Оставьте отзыв о скачивании PDF-файла.

Локальная документация по Foundry

Безопасное проектирование, настройка и управление приложениями ИИ и агентами на устройстве.

Узнайте о локальном литейном

ОБЗОР

Что такое Foundry Local?

КОНЦЕПЦИЯ

Локальная архитектура Foundry

Начало работы

КРАТКОЕ РУКОВОДСТВО

Начало работы с Foundry Local

ПРАКТИЧЕСКОЕ РУКОВОДСТВО

Интеграция с пакетами SDK для вывода

Интеграция с LangChain

Интеграция с открытым веб-интерфейсом

Компиляция моделей распознавания лиц для запуска в Foundry Local

Справка

ССЫЛКА

Справочник по интерфейсу командной строки (CLI)

Справочник по SDK

Справочник по REST

Справочник по API каталога

Рекомендации и устранение неполадок

Дополнительные ресурсы по ИИ

ОБУЧЕНИЕ

Обучение ИИ Azure

Векторный поиск в поиске ИИ Azure

ССЫЛКА

Azure AI Foundry

Семантическое ядро

Что такое Foundry Local?

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

Foundry Local — это решение для вывода AI непосредственно на устройстве, предлагающее преимущества в производительности, конфиденциальности, настройке и снижении затрат. Она легко интегрируется в существующие рабочие процессы и приложения с помощью интуитивно понятного интерфейса командной строки, пакета SDK и REST API.

Ключевые особенности

- **Инференция на устройстве**: выполняйте модели локально на собственном оборудовании, снижая затраты и сохраняя все данные на вашем устройстве.
- **Настройка модели**. Выберите из предустановленных моделей или используйте собственные для удовлетворения конкретных требований и вариантов использования.
- **Экономичность**. Устранение повторяющихся затрат на облачную службу с помощью существующего оборудования, что делает ИИ более доступным.
- **Простая интеграция**. Подключение к приложениям с помощью пакета SDK, конечных точек API или интерфейса командной строки с простым масштабированием в Azure AI Foundry по мере роста потребностей.

Случаи использования

Foundry Local идеально подходит для сценариев, в которых:

- Вы хотите сохранить конфиденциальные данные на устройстве.

- Вам нужно работать в средах с ограниченным доступом к Интернету или без подключения к Интернету.
- Вы хотите сократить затраты на вывод из облака.
- Для приложений в режиме реального времени требуются ответы искусственного интеллекта с низкой задержкой.
- Перед развертыванием в облачной среде необходимо поэкспериментировать с моделями ИИ.

Нужна ли подписка Azure?

Нет. Для локальной службы Foundry не требуется подписка Azure. Он выполняется на локальном оборудовании, что позволяет использовать существующую инфраструктуру без необходимости в облачных службах.

Начало работы

Установите и запустите первую модель, следуя руководству по началу работы с Foundry Local . Это руководство поможет вам настроить Foundry Local, обнаружить доступные модели и запустить первую локальную модель ИИ.

Дальнейшие шаги

- Начните работу с Foundry Local
- Как компилировать модели Hugging Face для запуска на Foundry Local

Начало работы с Foundry Local

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

В этом руководстве описано, как настроить Foundry Local для запуска моделей ИИ на устройстве.

Prerequisites

Система должна соответствовать следующим требованиям для запуска Foundry Local:

- **Операционная система**: Windows 10 (x64), Windows 11 (x64/ARM), Windows Server 2025, macOS.
- **Оборудование**: минимум 8 ГБ ОЗУ, 3 ГБ свободного места на диске. Рекомендуется 16 ГБ ОЗУ, свободное место на диске 15 ГБ.
- **Сеть**: подключение к Интернету для начальной загрузки модели (необязательно для автономного использования)
- **Ускорение (необязательно)**: GPU NVIDIA (2000 серии или более поздней версии), AMD GPU (6000 серии или более поздней версии), Intel iGPU, Qualcomm Snap nvidia X Elite (8 ГБ или больше памяти) или Apple Silicon.

Кроме того, убедитесь, что у вас есть права администратора для установки программного обеспечения на устройстве.

Quickstart

Начните быстро работать с Foundry Local:

1. **Установка Foundry Local**

    - **Windows**: откройте терминал и выполните следующую команду:

    ```
    Bash
    ```

```
winget install Microsoft.FoundryLocal
```

- **macOS**: откройте терминал и выполните следующую команду:

```bash
brew tap microsoft/foundrylocal
brew install foundrylocal
```

Кроме того, установщик можно скачать из репозитория Foundry Local GitHub.

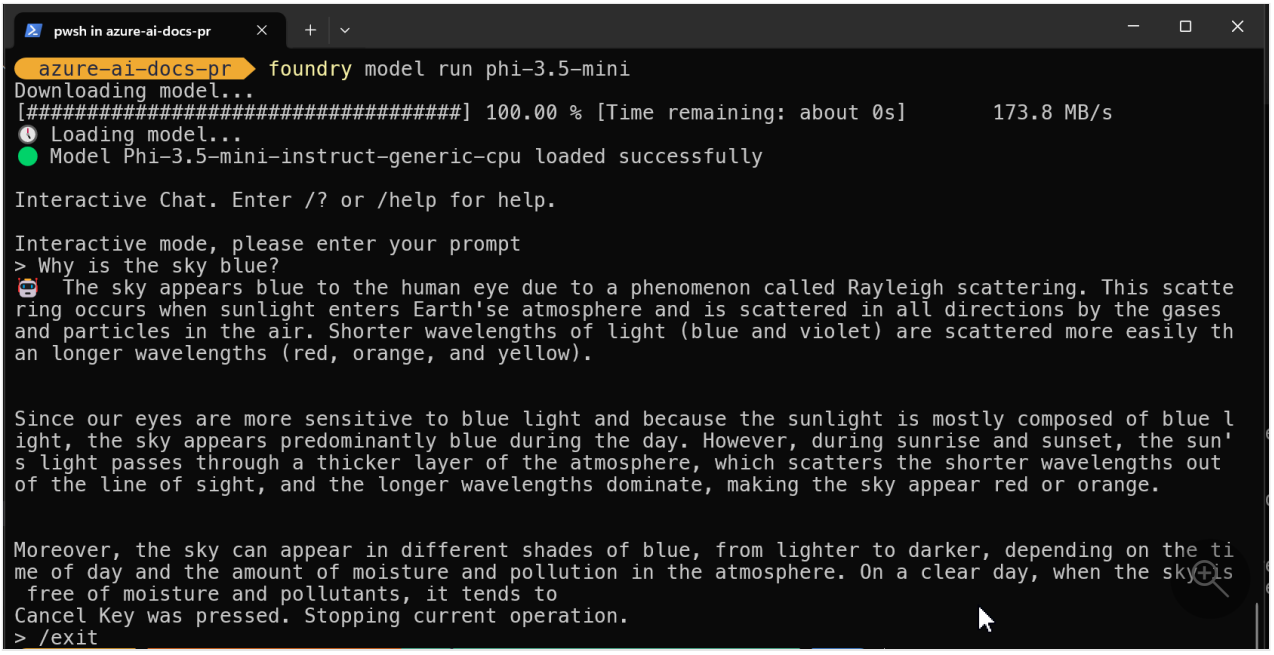
2. **Запуск первой модели** Откройте окно терминала и выполните следующую команду, чтобы запустить модель:

```bash
foundry model run phi-3.5-mini
```

Скачивание модели, которое может занять несколько минут в зависимости от скорости вашего интернета, и выполнение модели. После запуска модели вы можете взаимодействовать с ней с помощью интерфейса командной строки (CLI). Например, можно попросить:

```text
Why is the sky blue?
```

Вы должны увидеть ответ от модели в терминале:



Совет

Вы можете заменить `phi-3.5-mini` любое имя модели из каталога (см. сведения `foundry model list` о доступных моделях). Foundry Local скачивает вариант модели, который лучше всего соответствует конфигурации оборудования и программного обеспечения вашей системы. Например, если у вас есть GPU NVIDIA, он скачивает версию CUDA модели. Если у вас есть NPU (нейронный процессор) Qualcomm, система скачивает соответствующий вариант NPU. Если у вас нет GPU или NPU, он скачивает версию ЦП.

Запуск последней модели OpenAI с открытым кодом

Чтобы запустить последнюю модель OpenAI с открытым исходным кодом, `GPT-OSS-20B` используйте следующую команду:

```bash
foundry model run gpt-oss-20b
```

Важно!

Требования к запуску GPT-OSS-20B:

- Gpu Nvidia с 16 ГБ VRAM или более.
- Foundry Local версии **0.6.87** или более поздней. Любая версия ниже не будет поддерживать модель. Чтобы проверить локальную версию Foundry, выполните следующие действия:

```bash
foundry --version
```

Исследуйте команды

Интерфейс командной строки Foundry упорядочивает команды в следующие основные категории:

- **Модель**: команды для управления и запуска моделей.
- **Служба**: команды для управления локальной службой Foundry.
- **Кэш**: команды для управления локальным кэшем моделей (скачанные модели на локальном диске).

Просмотрите все доступные команды с помощью следующих команд:

```bash
foundry --help
```

Чтобы просмотреть доступные команды **модели** , выполните следующую команду:

```bash
foundry model --help
```

Чтобы просмотреть доступные команды **службы** , выполните следующую команду:

```bash
foundry service --help
```

Чтобы просмотреть доступные команды **кэша** , выполните следующую команду:

```bash
foundry cache --help
```

> **Совет**
>
> Полное руководство по всем командам CLI и их использованию см. в [справочнике по локальному интерфейсу командной строки Foundry](#).

Обновление локального литейного объекта

Чтобы обновить Foundry Local до последней версии, используйте следующие команды на основе операционной системы:

- **Windows**: откройте терминал и запустите:

  ```bash
  ```

```
winget upgrade --id Microsoft.FoundryLocal
```

- **macOS**: откройте терминал и выполните следующую команду:

```bash
brew upgrade foundrylocal
```

Удаление Foundry Local

Если вы хотите удалить Foundry Local, используйте следующие команды на основе операционной системы:

- **Windows**: откройте терминал и запустите:

```bash
winget uninstall Microsoft.FoundryLocal
```

- **macOS**: откройте терминал и выполните следующую команду:

```bash
brew rm foundrylocal
brew untap microsoft/foundrylocal
brew cleanup --scrub
```

Дальнейшие шаги

- Интегрировать SDK для инференса с Foundry Local
- Изучите локальную документацию по Foundry
- Сведения о рекомендациях и устранении неполадок
- Изучите справочник по локальному API Foundry
- Узнайте, как скомпилировать модели Hugging Face

Локальная архитектура Foundry

15.08.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

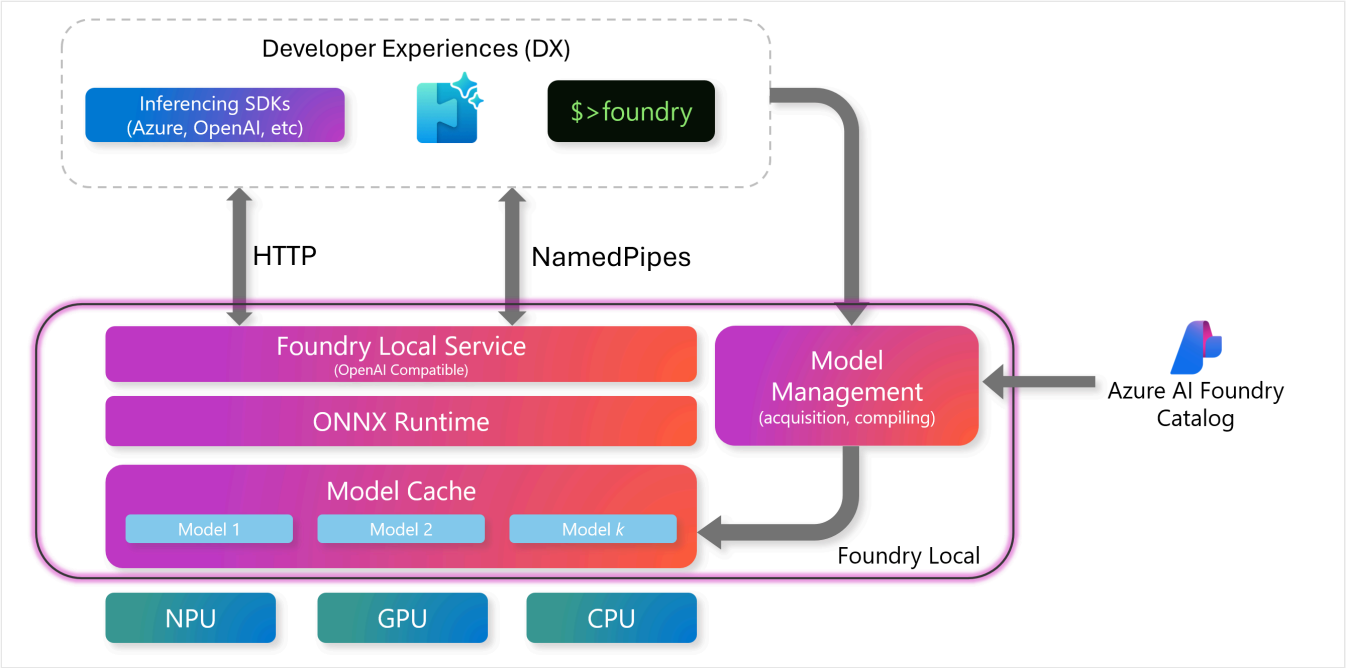
Foundry Local обеспечивает эффективный, безопасный и масштабируемый инференс модели ИИ непосредственно на устройствах. В этой статье описываются основные компоненты Foundry Local и их совместная работа для обеспечения возможностей искусственного интеллекта.

К ключевым преимуществам Foundry Local относятся:

- ✓ **Низкая задержка**: локально запустите модели, чтобы свести к минимуму время обработки и ускорить результаты.
- ✓ **Конфиденциальность данных**. Обработка конфиденциальных данных локально, не отправляя их в облако, помогая удовлетворить требования к защите данных.
- ✓ **Гибкость**. Поддержка различных конфигураций оборудования позволяет выбрать оптимальную настройку для ваших потребностей.
- ✓ **Масштабируемость**. Развертывание на разных устройствах, от ноутбуков до серверов, в соответствии с различными вариантами использования.
- ✓ **Экономичность**. Сокращение затрат на облачные вычисления, особенно для высокопроизводительных приложений.
- ✓ **Автономная работа**: работа без подключения к Интернету в отдаленных или изолированных средах.
- ✓ **Простая интеграция**: легко включить в существующие рабочие процессы разработки для плавного внедрения.

Ключевые компоненты

Локальная архитектура Foundry состоит из следующих основных компонентов:



Локальная служба Foundry

Локальная служба Foundry включает сервер REST, совместимый с OpenAI, который предоставляет стандартный интерфейс для работы с подсистемой вывода. Кроме того, можно управлять моделями через REST. Разработчики используют этот API для отправки запросов, запуска моделей и получения результатов программным способом.

- **Конечная точка: конечная** точка *динамически выделяется* при запуске службы. Конечную точку можно найти, выполнив `foundry service status` команду. При использовании Foundry Local в приложениях рекомендуется использовать пакет SDK, который автоматически обрабатывает конечную точку. Для получения дополнительных сведений о том, как использовать локальный SDK Foundry, прочитайте статью «Интеграция SDK для выводов с Foundry Local».
- **Варианты использования**:
  - Подключение Foundry Local к пользовательским приложениям
  - Выполнение моделей с помощью HTTP-запросов

Среда выполнения ONNX

Среда выполнения ONNX — это основной компонент, который выполняет модели искусственного интеллекта. Она эффективно работает с оптимизированными моделями ONNX на локальном оборудовании, например ЦП, GPU или NPUs.

Функции:

- Работает с несколькими поставщиками оборудования (NVIDIA, AMD, Intel, Qualcomm) и типами устройств (NPUs, ЦП, GPU)

- Предоставляет согласованный интерфейс для работы в разных моделях оборудования
- Обеспечивает лучшую производительность в классе
- Поддерживает квантизованные модели для ускорения вывода

управление моделями

Foundry Local предоставляет надежные инструменты для управления моделями искусственного интеллекта, обеспечивая доступность для вывода и простоты обслуживания. Управление моделями обрабатывается с помощью **кэша моделей** и **интерфейсаCommand-Line (CLI).**

Кэш моделей

Кэш моделей хранит скачанные модели ИИ локально на устройстве, что гарантирует, что модели готовы к выводу без необходимости скачать их повторно. Вы можете управлять кэшем с помощью интерфейса командной строки Foundry или REST API.

- **Назначение**. Ускорение вывода путем обеспечения локальной доступности моделей
- **Ключевые команды**:
  - `foundry cache list`: отображение всех моделей в локальном кэше
  - `foundry cache remove <model-name>`: удаляет определенную модель из кэша.
  - `foundry cache cd <path>`: изменяет расположение хранилища для кэшированных моделей

Жизненный цикл модели

1. **Скачайте** модели из каталога моделей Azure AI Foundry и сохраните их на локальный диск.
2. **Загрузка**: загрузка моделей в память локальной службы Foundry для инференции. Задайте TTL (время в реальном времени) для управления временем пребывания модели в памяти (по умолчанию: 10 минут).
3. **Запуск**: выполнение вывода модели для ваших запросов.
4. **Выгрузка**: удалите модели из памяти, чтобы освободить ресурсы, если они больше не нужны.
5. **Удаление**: Удалите модели из локального кэша для освобождения пространства на диске.

Компиляция моделей с помощью Оливки

Прежде чем модели можно использовать с Foundry Local, их необходимо скомпилировать и оптимизировать в формате ONNX. Корпорация Майкрософт предоставляет выбор опубликованных моделей в каталоге моделей Azure AI Foundry, которые уже оптимизированы для Foundry Local. Тем не менее, вы не ограничены этими моделями — благодаря Olive. Оливка — это мощная платформа для подготовки моделей искусственного интеллекта для эффективного вывода. Он преобразует модели в формат ONNX, оптимизирует структуру графа и применяет такие методы, как квантизация, чтобы повысить производительность локального оборудования.

> **Совет**
>
> Чтобы узнать больше о компиляции моделей для Foundry Local, прочитайте **Как компилировать модели Hugging Face для запуска на Foundry Local**.

Аппаратный уровень абстракции

Уровень абстракции оборудования гарантирует, что Foundry Local может работать на различных устройствах, абстрагируя базовое оборудование. Чтобы оптимизировать производительность на основе доступного оборудования, Foundry Local поддерживает:

- **несколько *поставщиков выполнения***, таких как NVIDIA CUDA, AMD, Qualcomm, Intel.
- **несколько *типов устройств***, таких как ЦП, GPU, NPU.

Возможности разработчика

Локальная архитектура Foundry предназначена для обеспечения простого интерфейса разработчика, что позволяет легко интегрировать и взаимодействовать с моделями искусственного интеллекта. Разработчики могут выбирать различные интерфейсы для взаимодействия с системой, в том числе:

интерфейс Command-Line (CLI)

Cli Foundry — это мощный инструмент для управления моделями, подсистемой вывода и локальным кэшем.

Примеры:

- `foundry model list`: перечисляет все доступные модели в локальном кэше.
- `foundry model run <model-name>`: выполняет модель.
- `foundry service status`: проверяет состояние службы.

> **Совет**
>
> Дополнительные сведения о командах CLI см. в **справочнике по локальному интерфейсу командной строки Foundry**.

Интеграция пакета SDK для вывода

Foundry Local поддерживает интеграцию с различными пакетами SDK на большинстве языков, таких как пакет SDK OpenAI, что позволяет разработчикам использовать знакомые интерфейсы программирования для взаимодействия с локальным механизмом вывода.

> **Совет**
>
> Чтобы узнать больше об интеграции с пакетами SDK для вывода, прочтите **«Интеграция с пакетами SDK для вывода в Foundry Local»**.

Набор средств ИИ для Visual Studio Code

Набор средств ИИ для Visual Studio Code предоставляет удобный для разработчиков интерфейс для взаимодействия с Foundry Local. Он позволяет пользователям запускать модели, управлять локальным кэшем и визуализировать результаты непосредственно в интегрированной среде разработки.

Функции:

- Управление моделями: загрузка, загрузка и запуск моделей из интегрированной среды разработки.
- Интерактивная консоль: отправка запросов и просмотр ответов в режиме реального времени.
- Средства визуализации: графическое представление производительности и результатов модели.

Необходимые условия:

- Вы установили Foundry Local и у вас запущена служба моделей.
- Вы установили расширение AI Toolkit для Visual Studio Code .

Подключение локальной модели Foundry к набору средств искусственного интеллекта:

1. **Добавьте модель в ai Toolkit: Open AI Toolkit** из панели действий Visual Studio Code. На панели "Мои модели" нажмите кнопку "Добавить модель для удаленного интерфейса" и выберите "Добавить пользовательскую модель" в раскрывающемся меню.
2. **Введите URL-адрес конечной точки, совместимой с чатом: введите** `http://localhost:PORT/v1/chat/completions`, где ПОРТ заменяется номером порта конечной точки локальной службы Foundry. Вы можете увидеть порт локально работающей службы с помощью команды `foundry service status` CLI. Foundry Local динамически назначает порт, поэтому он может не всегда совпадать.
3. **Укажите имя модели**: введите точное имя модели, которое следует использовать из Foundry Local, например `phi-3.5-mini`. Вы можете перечислить все ранее скачанные и локально кэшированные модели с помощью команды `foundry cache list` CLI или использовать `foundry model list` для просмотра всех доступных моделей для локального использования. Вам также будет предложено ввести отображаемое имя, которое предназначено только для собственного локального использования, поэтому, чтобы избежать путаницы, рекомендуется ввести то же имя, что и точное имя модели.
4. **Проверка подлинности**. Если локальная настройка не требует проверки подлинности *(которая используется по умолчанию для локальной установки Foundry),* можно оставить поле заголовков проверки подлинности пустым и нажмите клавишу ВВОД.

После выполнения этих действий локальная модель Foundry появится в списке "Мои модели" в средстве искусственного интеллекта и будет готова к использованию, щелкнув правой кнопкой мыши модель и выбрав "Загрузить на площадке".

Дальнейшие шаги

- Начните работу с Foundry Local
- Интегрировать SDK для инференса с Foundry Local
- Справочник по локальному интерфейсу командной строки Foundry

Интегрируйте SDK для инференса с Foundry Local

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

Foundry Local интегрируется с различными пакетами SDK для вывода, такими как OpenAI, Azure OpenAI, Langchain и т. д. В этом руководстве показано, как подключить приложения к локальным моделям ИИ с помощью популярных пакетов SDK.

Предпосылки

- Программа Foundry Local установлена. Инструкции по установке см. в статье "Начало работы с Foundry Local ".

Установка пакетов Node.js

Необходимо установить следующие пакеты Node.js:

```bash
npm install openai
npm install foundry-local-sdk
```

Локальный пакет SDK Foundry позволяет управлять локальной службой и моделями Foundry.

Использование пакета SDK OpenAI с локальным пакетом Foundry

В следующем примере показано, как использовать пакет SDK OpenAI с Foundry Local. Код инициализирует локальную службу Foundry, загружает модель и создает ответ с помощью пакета SDK OpenAI.

Скопируйте и вставьте следующий код в файл JavaScript с именем `app.js`:

JavaScript

```javascript
import { OpenAI } from "openai";
import { FoundryLocalManager } from "foundry-local-sdk";

// By using an alias, the most suitable model will be downloaded
// to your end-user's device.
// TIP: You can find a list of available models by running the
// following command in your terminal: `foundry model list`.
const alias = "phi-3.5-mini";

// Create a FoundryLocalManager instance. This will start the Foundry
// Local service if it is not already running.
const foundryLocalManager = new FoundryLocalManager()

// Initialize the manager with a model. This will download the model
// if it is not already present on the user's device.
const modelInfo = await foundryLocalManager.init(alias)
console.log("Model Info:", modelInfo)

const openai = new OpenAI({
  baseURL: foundryLocalManager.endpoint,
  apiKey: foundryLocalManager.apiKey,
});

async function generateText() {
  const response = await openai.chat.completions.create({
    model: modelInfo.id,
    messages: [
      {
        role: "user",
        content: "What is the golden ratio?",
      },
    ],
  });

  console.log(response.choices[0].message.content);
}

generateText();
```

Запустите код с помощью следующей команды:

Bash

```
node app.js
```

Стриминговые ответы

Если вы хотите получать ответы потоковой передачи, измените код так:

JavaScript

```javascript
import { OpenAI } from "openai";
import { FoundryLocalManager } from "foundry-local-sdk";

// By using an alias, the most suitable model will be downloaded
// to your end-user's device.
// TIP: You can find a list of available models by running the
// following command in your terminal: `foundry model list`.
const alias = "phi-3.5-mini";

// Create a FoundryLocalManager instance. This will start the Foundry
// Local service if it is not already running.
const foundryLocalManager = new FoundryLocalManager()

// Initialize the manager with a model. This will download the model
// if it is not already present on the user's device.
const modelInfo = await foundryLocalManager.init(alias)
console.log("Model Info:", modelInfo)

const openai = new OpenAI({
  baseURL: foundryLocalManager.endpoint,
  apiKey: foundryLocalManager.apiKey,
});

async function streamCompletion() {
    const stream = await openai.chat.completions.create({
      model: modelInfo.id,
      messages: [{ role: "user", content: "What is the golden ratio?" }],
      stream: true,
    });

    for await (const chunk of stream) {
      if (chunk.choices[0]?.delta?.content) {
        process.stdout.write(chunk.choices[0].delta.content);
      }
    }
}

streamCompletion();
```

Запустите код с помощью следующей команды:

Bash

```bash
node app.js
```

Используйте Fetch API с Foundry Local

Если вы предпочитаете использовать HTTP-клиент, например `fetch`, можно сделать следующее:

JavaScript

```javascript
import { FoundryLocalManager } from "foundry-local-sdk";

// By using an alias, the most suitable model will be downloaded
// to your end-user's device.
// TIP: You can find a list of available models by running the
// following command in your terminal: `foundry model list`.
const alias = "phi-3.5-mini";

// Create a FoundryLocalManager instance. This will start the Foundry
// Local service if it is not already running.
const foundryLocalManager = new FoundryLocalManager()

// Initialize the manager with a model. This will download the model
// if it is not already present on the user's device.
const modelInfo = await foundryLocalManager.init(alias)
console.log("Model Info:", modelInfo)

async function queryModel() {
    const response = await fetch(foundryLocalManager.endpoint +
"/chat/completions", {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
        },
        body: JSON.stringify({
            model: modelInfo.id,
            messages: [
                { role: "user", content: "What is the golden ratio?" },
            ],
        }),
    });

    const data = await response.json();
    console.log(data.choices[0].message.content);
}

queryModel();
```

Стриминговые ответы

Если вы хотите получать ответы в потоковом режиме с помощью Fetch API, можно изменить код следующим образом:

JavaScript

```javascript
import { FoundryLocalManager } from "foundry-local-sdk";

// By using an alias, the most suitable model will be downloaded
// to your end-user's device.
// TIP: You can find a list of available models by running the
// following command in your terminal: `foundry model list`.
const alias = "phi-3.5-mini";

// Create a FoundryLocalManager instance. This will start the Foundry
// Local service if it is not already running.
const foundryLocalManager = new FoundryLocalManager()

// Initialize the manager with a model. This will download the model
// if it is not already present on the user's device.
const modelInfo = await foundryLocalManager.init(alias)
console.log("Model Info:", modelInfo)

async function streamWithFetch() {
    const response = await fetch(foundryLocalManager.endpoint +
"/chat/completions", {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
            Accept: "text/event-stream",
        },
        body: JSON.stringify({
            model: modelInfo.id,
            messages: [{ role: "user", content: "what is the golden ratio?" }],
            stream: true,
        }),
    });

    const reader = response.body.getReader();
    const decoder = new TextDecoder();

    while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        const chunk = decoder.decode(value);
        const lines = chunk.split("\n").filter((line) => line.trim() !== "");

        for (const line of lines) {
            if (line.startsWith("data: ")) {
                const data = line.substring(6);
                if (data === "[DONE]") continue;
```

```
                try {
                    const json = JSON.parse(data);
                    const content = json.choices[0]?.delta?.content || "";
                    if (content) {
                        // Print to console without line breaks, similar to
process.stdout.write
                        process.stdout.write(content);
                    }
                } catch (e) {
                    console.error("Error parsing JSON:", e);
                }
            }
        }
    }
}

// Call the function to start streaming
streamWithFetch();
```

Дальнейшие шаги

- Компилировать модели Hugging Face для запуска в Foundry Local
- Изучите справочник по локальному интерфейсу командной строки Foundry

Создание приложения перевода с помощью LangChain

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

В этом руководстве показано, как создать приложение с помощью локального пакета SDK Foundry и LangChain. В этом руководстве вы создадите приложение перевода, которое преобразует текст с одного языка на другой, использующий локальную модель.

Предпосылки

Перед началом работы с этим руководством вам потребуется:

- На компьютере установлено **локальное приложение Foundry**. Ознакомьтесь с руководством по началу работы с Foundry Local для инструкций по установке.
- **Node.js 18 или более поздняя версия** установлена на компьютере. Вы можете скачать Node.js с официального веб-сайта.

Установка пакетов Node.js

Необходимо установить следующие пакеты Node.js:

```bash
npm install @langchain/openai @langchain/core
npm install foundry-local-sdk
```

Создание приложения перевода

Создайте новый файл JavaScript с именем `translation_app.js` в любимой интегрированной среде разработки и добавьте следующий код:

JavaScript

```
import { FoundryLocalManager } from "foundry-local-sdk";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";

// By using an alias, the most suitable model will be downloaded
// to your end-user's device.
// TIP: You can find a list of available models by running the
// following command in your terminal: `foundry model list`.
const alias = "phi-3-mini-4k";

// Create a FoundryLocalManager instance. This will start the Foundry
// Local service if it is not already running.
const foundryLocalManager = new FoundryLocalManager()

// Initialize the manager with a model. This will download the model
// if it is not already present on the user's device.
const modelInfo = await foundryLocalManager.init(alias)
console.log("Model Info:", modelInfo)

// Configure ChatOpenAI to use your locally-running model
const llm = new ChatOpenAI({
    model: modelInfo.id,
    configuration: {
        baseURL: foundryLocalManager.endpoint,
        apiKey: foundryLocalManager.apiKey
    },
    temperature: 0.6,
    streaming: false
});

// Create a translation prompt template
const prompt = ChatPromptTemplate.fromMessages([
    {
        role: "system",
        content: "You are a helpful assistant that translates {input_language} to
{output_language}."
    },
    {
        role: "user",
        content: "{input}"
    }
]);

// Build a simple chain by connecting the prompt to the language model
const chain = prompt.pipe(llm);

const input = "I love to code.";
console.log(`Translating '${input}' to French...`);
```

```
// Run the chain with your inputs
chain.invoke({
    input_language: "English",
    output_language: "French",
    input: input
}).then(aiMsg => {
    // Print the result content
    console.log(`Response: ${aiMsg.content}`);
}).catch(err => {
    console.error("Error:", err);
});
```

> ⓘ **Примечание**
>
> Одним из ключевых преимуществ Foundry Local является то, что он **автоматически** выбирает наиболее подходящий **вариант** модели для оборудования пользователя. Например, если у пользователя есть GPU, он скачивает версию GPU модели. Если у пользователя есть NPU (единица нейронной обработки), он скачивает версию NPU. Если у пользователя нет GPU или NPU, он скачивает версию ЦП модели.

Запуск приложения

Чтобы запустить приложение, откройте терминал и перейдите в каталог, в котором вы сохранили `translation_app.js` файл. Затем выполните следующую команду:

```bash
node translation_app.js
```

Дальнейшие шаги

- Ознакомьтесь с документацией по LangChain для более сложных функций и возможностей.
- Компилировать модели Hugging Face для запуска в Foundry Local

Интеграция открытого веб-интерфейса с Foundry Local

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

В этом руководстве показано, как создать приложение чата с помощью Foundry Local и Open Web UI. По завершении работы у вас есть рабочий интерфейс чата, работающий полностью на локальном устройстве.

Предпосылки

Прежде чем приступить к работе с этим руководством, вам потребуется:

- На компьютере установлено **локальное приложение Foundry**. Ознакомьтесь с руководством по началу работы с Foundry Local для инструкций по установке.

Настройка открытого веб-интерфейса для чата

1. **Установите открытый веб-интерфейс** , следуя инструкциям из репозитория GitHub открытого веб-интерфейса .

2. **Запустите открытый веб-интерфейс** с помощью этой команды в терминале:

   Bash
   ```
   open-webui serve
   ```

3. Откройте веб-браузер и перейдите по адресу http://localhost:8080.

4. **Подключение открытого веб-интерфейса к Foundry Local**:
   a. Выбор **параметров** в меню навигации
   b. Выбор **подключений**

c. Выбор "**Управление прямыми подключениями**"
d. + Выберите значок для добавления подключения
e. В поле **URL-адреса** введите `http://localhost:PORT/v1` , где `PORT` заменяется порт локальной конечной точки Foundry, который можно найти с помощью команды `foundry service status` CLI. Обратите внимание, что Foundry Local динамически назначает порт, поэтому он не всегда совпадает.
f. Введите любое значение (например) для ключа API, так как `test` оно не может быть пустым.
g. Сохранение подключения

5. **Начните чат с моделью**:
   a. Загруженные модели отображаются в раскрывающемся списке в верхней части
   b. Выберите любую модель из списка
   c. Введите сообщение в поле ввода внизу

Вот и все! Теперь вы общаетесь с моделью искусственного интеллекта, работающей полностью на локальном устройстве.

Дальнейшие шаги

- Интегрировать SDK для инференса с Foundry Local
- Компилировать модели Hugging Face для запуска в Foundry Local

Компиляция моделей распознавания лиц для запуска в Foundry Local

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

Foundry Local запускает модели ONNX на устройстве с высокой производительностью. Хотя каталог моделей предлагает готовые заранее скомпилированные опции, вы можете использовать любую модель в формате ONNX.

Чтобы скомпилировать существующие модели в формате Safetensor или PyTorch в формате ONNX, можно использовать Оливку. Оливка — это средство, которое оптимизирует модели в формат ONNX, что делает их подходящими для развертывания в Foundry Local. В нем используются такие методы, как *квантизация* и *оптимизация графа* для повышения производительности.

В этом руководстве показано, как:

- ✓ **Преобразование и оптимизация** моделей из Hugging Face для запуска в Foundry Local. Вы будете `Llama-3.2-1B-Instruct` использовать модель в качестве примера, но вы можете использовать любую созданную модель искусственного интеллекта из Hugging Face.
- ✓ **Запуск** оптимизированных моделей с помощью Foundry Local

Предпосылки

- Python 3.10 или более поздней версии

Установите Olive

Оливка — это средство, оптимизирующее модели в формате ONNX.

Баш

```bash
pip install olive-ai[auto-opt]
```

Совет

Для получения наилучших результатов установите Олив в виртуальной среде с помощью venv или conda.

Вход в hugging Face

Вы оптимизируете `Llama-3.2-1B-Instruct` модель, для которой требуется проверка подлинности лиц Hugging:

Баш

```bash
huggingface-cli login
```

Примечание

Прежде чем продолжить, необходимо создать маркер распознавания лиц и запросить доступ к модели .

Компиляция модели

Шаг 1. Выполните команду "Оливковая автоматическая выборка"

Используйте команду Olive `auto-opt` для скачивания, преобразования, квантизации и оптимизации модели:

Баш

```bash
olive auto-opt \
    --model_name_or_path meta-llama/Llama-3.2-1B-Instruct \
    --trust_remote_code \
    --output_path models/llama \
    --device cpu \
    --provider CPUExecutionProvider \
    --use_ort_genai \
    --precision int4 \
    --log_level 1
```

Примечание

Процесс компиляции занимает около 60 секунд, а также дополнительное время для скачивания модели.

Команда использует следующие параметры:

Развернуть таблицу

Параметр	Описание
`model_name_or_path`	Источник модели: Hugging Face ID, локальный путь или идентификатор реестра моделей Azure AI
`output_path`	Где сохранить оптимизированную модель
`device`	Целевое оборудование: `cpu`, `gpu` или `npu`
`provider`	Поставщик выполнения (например, `CPUExecutionProvider`, `CUDAExecutionProvider`)
`precision`	Точность модели: `fp16`, , `int4` `fp32` или `int8`
`use_ort_genai`	Создает файлы конфигурации вывода

Совет

Если у вас есть локальная копия модели, можно использовать локальный путь вместо идентификатора лица обнимания. Например: `--model_name_or_path models/llama-3.2-`

`1B-Instruct`. Оливка автоматически обрабатывает преобразование, оптимизацию и квантизацию.

Шаг 2. Переименование выходной модели

Оливка помещает файлы в универсальный `model` каталог. Переименуйте его, чтобы упростить использование:

Баш

Bash

```bash
cd models/llama
mv model llama-3.2
```

Шаг 3. Создание файла шаблона чата

Шаблон чата — это структурированный формат, определяющий способ обработки входных и выходных сообщений для модели ИИ беседы. Он задает роли (например, системные, пользовательские, помощники) и структуру беседы, гарантируя, что модель понимает контекст и создает соответствующие ответы.

Foundry Local требует json-файл шаблона чата `inference_model.json`, чтобы создавать соответствующие ответы. Свойства шаблона — это имя модели и `PromptTemplate` объект, который содержит заполнитель, который Foundry Local внедряет `{Content}` во время выполнения с помощью запроса пользователя.

JSON

```json
{
  "Name": "llama-3.2",
  "PromptTemplate": {
    "assistant": "{Content}",
    "prompt": "<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>\n\nCutting Knowledge Date: December
2023\nToday Date: 26 Jul 2024\n\nYou are a helpful assistant.<|eot_id|>
<|start_header_id|>user<|end_header_id|>\n\n{Content}<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>\n\n"
  }
}
```

Чтобы создать файл шаблона чата, можно использовать `apply_chat_template` метод из библиотеки hugging Face:

> ⓘ **Примечание**
>
> В следующем примере для создания шаблона чата используется библиотека распознавания лиц Python Hugging. Библиотека Hugging Face является зависимостью для Olive, поэтому если вы используете ту же виртуальную среду Python, вам не нужно её устанавливать. Если вы используете другую среду, установите библиотеку с `pip install transformers` помощью .

Python

```python
# generate_inference_model.py
# This script generates the inference_model.json file for the Llama-3.2 model.
import json
import os
from transformers import AutoTokenizer

model_path = "models/llama/llama-3.2"

tokenizer = AutoTokenizer.from_pretrained(model_path)
chat = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "{Content}"},
]


template = tokenizer.apply_chat_template(chat, tokenize=False,
add_generation_prompt=True)

json_template = {
  "Name": "llama-3.2",
  "PromptTemplate": {
    "assistant": "{Content}",
    "prompt": template
  }
}

json_file = os.path.join(model_path, "inference_model.json")

with open(json_file, "w") as f:
    json.dump(json_template, f, indent=2)
```

Запустите скрипт с помощью:

Bash

```
python generate_inference_model.py
```

Запуск модели

Вы можете запустить скомпилированную модель с помощью локального интерфейса командной строки Foundry, REST API или пакета SDK для OpenAI Python. Сначала измените каталог кэша моделей на каталог моделей, созданный на предыдущем шаге:

Bash

Bash

```bash
foundry cache cd models
foundry cache ls  # should show llama-3.2
```

⊗ Внимание!

Не забудьте вернуть кэш модели обратно в каталог по умолчанию после выполнения.

Bash

```bash
foundry cache cd ./foundry/cache/models.
```

Использование локального интерфейса командной строки Foundry

Баш

Bash

```bash
foundry model run llama-3.2 --verbose
```

Использование пакета SDK Для Python OpenAI

Пакет SDK Для Python OpenAI — это удобный способ взаимодействия с локальным REST API Foundry. Его можно установить с помощью:

Bash

```bash
pip install openai
pip install foundry-local-sdk
```

Затем можно использовать следующий код для запуска модели:

Python

```python
import openai
from foundry_local import FoundryLocalManager

modelId = "llama-3.2"

# Create a FoundryLocalManager instance. This will start the Foundry
# Local service if it is not already running and load the specified model.
manager = FoundryLocalManager(modelId)

# The remaining code us es the OpenAI Python SDK to interact with the local model.

# Configure the client to use the local Foundry service
client = openai.OpenAI(
    base_url=manager.endpoint,
    api_key=manager.api_key  # API key is not required for local usage
)

# Set the model to use and generate a streaming response
stream = client.chat.completions.create(
    model=manager.get_model_info(modelId).id,
    messages=[{"role": "user", "content": "What is the golden ratio?"}],
    stream=True
)

# Print the streaming response
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
```

Совет

Вы можете использовать любой язык, поддерживающий HTTP-запросы. Дополнительную информацию см. в статье [О SDK для интеграции интеллектуального вывода с Foundry Local](#).

Завершение

После завершения работы с пользовательской моделью необходимо сбросить кэш модели в каталог по умолчанию с помощью:

```bash
foundry cache cd ./foundry/cache/models
```

Дальнейшие шаги

- Дополнительные сведения о Оливе
- Интегрировать SDK для инференса с Foundry Local

Справочник по локальному интерфейсу командной строки Foundry

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

В этой статье содержится исчерпывающая справка по интерфейсу командной строки Foundry Local (CLI). Интерфейс командной строки упорядочивает команды в логические категории, помогая управлять моделями, управлять службой и поддерживать локальный кэш.

Обзор

Просмотрите все доступные команды с помощью опции помощи.

```bash
foundry --help
```

Интерфейс командной строки упорядочивает команды в три основные категории:

- **Модель**: команды для управления и запуска моделей ИИ
- **Служба**: команды для управления локальной службой Foundry
- **Кэш**: команды для управления локальным хранилищем моделей

Команды модели

В следующей таблице приведены команды, связанные с управлением и выполнением моделей:

> ⓘ **Примечание**

Аргумент можно указать по `model` или **идентификатору модели**. Использование псевдонима позволит:

- Выберите *лучшую модель* для вашего доступного оборудования. Например, если у вас есть gpu Nvidia CUDA, Foundry Local выбирает модель CUDA. Если у вас есть поддерживаемый NPU, Foundry Local выбирает модель NPU.
- Позволяет использовать более короткое имя без необходимости запоминать идентификатор модели.

Если вы хотите запустить определенную модель, можно использовать идентификатор модели. Например, чтобы запустить `qwen2.5-0.5b` на процессоре, независимо от наличия другого оборудования, используйте: `foundry model run qwen2.5-0.5b-instruct-generic-cpu`.

Развернуть таблицу

Команда	Описание
`foundry model --help`	Отображает все доступные команды, связанные с моделью, и их использование.
`foundry model run <model>`	Запускает указанную модель, скачивая ее, если она не кэширована, и запускает взаимодействие.
`foundry model list`	Выводит список всех доступных моделей для локального использования.
`foundry model info <model>`	Отображает подробные сведения о конкретной модели.
`foundry model info <model> --license`	Отображает сведения о лицензии для конкретной модели.
`foundry model download <model>`	Загружает модель в локальный кэш без его запуска.
`foundry model load <model>`	Загружает модель в службу.
`foundry model unload <model>`	Выгружает модель из службы.

Команды сервиса

В следующей таблице приведены команды, связанные с управлением и запуском локальной службы Foundry:

Развернуть таблицу

Команда	Описание
`foundry service --help`	Отображает все доступные команды, связанные со службой, и их использование.
`foundry service start`	Запускает локальную службу Foundry.
`foundry service stop`	Останавливает локальную службу Foundry.
`foundry service restart`	Перезапуск локальной службы Foundry.
`foundry service status`	Отображает текущее состояние локальной службы Foundry.
`foundry service ps`	Выводит список всех моделей, загруженных в локальную службу Foundry.
`foundry service diag`	Отображает журналы локальной службы Foundry.
`foundry service set <options>`	Задайте конфигурацию локальной службы Foundry.

Команды кэша

В следующей таблице приведены команды, связанные с управлением локальным кэшем, в котором хранятся модели:

Развернуть таблицу

Команда	Описание
`foundry cache --help`	Отображает все доступные команды, связанные с кэшем, и их использование.
`foundry cache location`	Отображает текущий каталог кэша.
`foundry cache list`	Выводит список всех моделей, хранящихся в локальном кэше.
`foundry cache cd <path>`	Изменяет каталог кэша.
`foundry cache remove <model>`	Удаляет модель из локального кэша.

Справочник по локальному пакету SDK для Foundry

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

Локальный пакет SDK Foundry упрощает управление моделями ИИ в локальных средах, предоставляя операции уровня управления отдельно от кода вывода на уровне данных. В этом справочнике описана реализация пакета SDK для Python, JavaScript, C# и Rust.

Справочник по пакету SDK для JavaScript

Установка

Установите пакет из npm:

```bash
npm install foundry-local-sdk
```

Класс FoundryLocalManager

Класс `FoundryLocalManager` позволяет управлять моделями, управлять кэшем и взаимодействовать с локальной службой Foundry в браузерах и Node.js средах.

Инициализация

```javascript
import { FoundryLocalManager } from "foundry-local-sdk";
```

```
const foundryLocalManager = new FoundryLocalManager()
```

Доступные варианты:

- `serviceUrl`: базовый URL-адрес локальной службы Foundry
- `fetch`: (необязательно) Настраиваемая функция извлечения для таких сред, как Node.js

Примечание по псевдонимам

Многие методы, описанные в этой ссылке, имеют `aliasOrModelId` параметр в сигнатуре. Можно передать **в метод псевдоним** или **идентификатор модели** в качестве значения. Использование псевдонима позволит:

- Выберите *лучшую модель* для доступного оборудования. Например, если gpu Nvidia CUDA доступен, Foundry Local выбирает модель CUDA. Если доступен поддерживаемый NPU, Foundry Local выбирает модель NPU.
- Позволяет использовать более короткое имя без необходимости запоминать идентификатор модели.

> **Совет**
>
> Рекомендуется передать в параметр `aliasOrModelId` **псевдоним**, так как при развертывании приложения Foundry Local получает наилучшую модель для компьютера конечного пользователя в режиме выполнения.

Управление службой

Развернуть таблицу

Метод	Подпись	Описание
`init()`	`(aliasOrModelId?: string) => Promise<void>`	Инициализирует пакет SDK и при необходимости загружает модель.
`isServiceRunning()`	`() => Promise<boolean>`	Проверяет, запущена ли локальная служба Foundry.
`startService()`	`() => Promise<void>`	Запускает локальную службу Foundry.
`serviceUrl`	`string`	Базовый URL-адрес локальной службы Foundry.

Метод	Подпись	Описание
`endpoint`	`string`	Конечная точка API (serviceUrl + `/v1`).
`apiKey`	`string`	Ключ API (нет).

Управление каталогами

Развернуть таблицу

Метод	Подпись	Описание
`listCatalogModels()`	`() => Promise<FoundryModelInfo[]>`	Выводит список всех доступных моделей в каталоге.
`refreshCatalog()`	`() => Promise<void>`	Обновляет каталог моделей.
`getModelInfo()`	`(aliasOrModelId: string, throwOnNotFound = false) => Promise<FoundryModelInfo | null>`	Возвращает сведения о модели по псевдониму или идентификатору.

Управление кэшем

Развернуть таблицу

Метод	Подпись	Описание
`getCacheLocation()`	`() => Promise<string>`	Возвращает путь к каталогу кэша моделей.
`listCachedModels()`	`() => Promise<FoundryModelInfo[]>`	Выводит список моделей, скачанных в локальный кэш.

Управление моделями

Развернуть таблицу

Метод	Подпись	Описание
`downloadModel()`	`(aliasOrModelId: string, token?: string, force = false, onProgress?) => Promise<FoundryModelInfo>`	Загружает модель в локальный кэш.
`loadModel()`	`(aliasOrModelId: string, ttl = 600) => Promise<FoundryModelInfo>`	Загружает модель на сервер вывода.

Метод	Подпись	Описание
`unloadModel()`	`(aliasOrModelId: string, force = false) => Promise<void>`	Выгружает модель с сервера вывода.
`listLoadedModels()`	`() => Promise<FoundryModelInfo[]>`	Выводит список всех моделей, загруженных в настоящее время в сервисе.

Пример использования

В следующем коде показано, как использовать `FoundryLocalManager` класс для управления моделями и взаимодействия с локальной службой Foundry.

JavaScript

```
import { FoundryLocalManager } from "foundry-local-sdk";

// By using an alias, the most suitable model will be downloaded
// to your end-user's device.
// TIP: You can find a list of available models by running the
// following command in your terminal: `foundry model list`.
const alias = "phi-3.5-mini";

const manager = new FoundryLocalManager()

// Initialize the SDK and optionally load a model
const modelInfo = await manager.init(alias)
console.log("Model Info:", modelInfo)

// Check if the service is running
const isRunning = await manager.isServiceRunning()
console.log(`Service running: ${isRunning}`)

// List available models in the catalog
const catalog = await manager.listCatalogModels()

// Download and load a model
await manager.downloadModel(alias)
await manager.loadModel(alias)

// List models in cache
const localModels = await manager.listCachedModels()

// List loaded models
const loaded = await manager.listLoadedModels()

// Unload a model
await manager.unloadModel(alias)
```

Интеграция с клиентом OpenAI

Установите пакет OpenAI:

Bash

```bash
npm install openai
```

В следующем коде показано, как интегрировать `FoundryLocalManager` с клиентом OpenAI для взаимодействия с локальной моделью.

JavaScript

```javascript
import { OpenAI } from "openai";
import { FoundryLocalManager } from "foundry-local-sdk";

// By using an alias, the most suitable model will be downloaded
// to your end-user's device.
// TIP: You can find a list of available models by running the
// following command in your terminal: `foundry model list`.
const alias = "phi-3.5-mini";

// Create a FoundryLocalManager instance. This will start the Foundry
// Local service if it is not already running.
const foundryLocalManager = new FoundryLocalManager()

// Initialize the manager with a model. This will download the model
// if it is not already present on the user's device.
const modelInfo = await foundryLocalManager.init(alias)
console.log("Model Info:", modelInfo)

const openai = new OpenAI({
  baseURL: foundryLocalManager.endpoint,
  apiKey: foundryLocalManager.apiKey,
});

async function streamCompletion() {
    const stream = await openai.chat.completions.create({
      model: modelInfo.id,
      messages: [{ role: "user", content: "What is the golden ratio?" }],
      stream: true,
    });

    for await (const chunk of stream) {
      if (chunk.choices[0]?.delta?.content) {
        process.stdout.write(chunk.choices[0].delta.content);
      }
    }
}
```

```
streamCompletion();
```

Использование браузера

Пакет SDK включает версию, совместимую с браузером, где необходимо указать URL-адрес службы вручную:

JavaScript

```javascript
import { FoundryLocalManager } from "foundry-local-sdk/browser"

// Specify the service URL
// Run the Foundry Local service using the CLI: `foundry service start`
// and use the URL from the CLI output
const endpoint = "ENDPOINT"

const manager = new FoundryLocalManager({serviceUrl: endpoint})

// Note: The `init`, `isServiceRunning`, and `startService` methods
// are not available in the browser version
```

Примечание

Версия браузера не поддерживает методы `init`, `isServiceRunning` и `startService`. Перед использованием пакета SDK в среде браузера необходимо убедиться, что локальная служба Foundry запущена. Вы можете запустить службу с помощью локального интерфейса командной строки Foundry: `foundry service start`. Url-адрес службы можно получить из выходных данных ИНТЕРФЕЙСА командной строки.

Пример использования

JavaScript

```javascript
import { FoundryLocalManager } from "foundry-local-sdk/browser"

// Specify the service URL
// Run the Foundry Local service using the CLI: `foundry service start`
// and use the URL from the CLI output
const endpoint = "ENDPOINT"

const manager = new FoundryLocalManager({serviceUrl: endpoint})

const alias = 'phi-3.5-mini'
```

```
// Get all available models
const catalog = await manager.listCatalogModels()
console.log("Available models in catalog:", catalog)

// Download and load a specific model
await manager.downloadModel(alias)
await manager.loadModel(alias)

// View models in your local cache
const localModels = await manager.listLocalModels()
console.log("Cached models:", catalog)

// Check which models are currently loaded
const loaded = await manager.listLoadedModels()
console.log("Loaded models in inference service:", loaded)

// Unload a model when finished
await manager.unloadModel(alias)
```

Справочник по локальному REST API Foundry

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

> ⊗ **Внимание!**
>
> Этот API находится в активной разработке и может включать критические изменения без уведомления. Мы настоятельно рекомендуем отслеживать журнал изменений перед созданием рабочих приложений.

Совместимость OpenAI версии 1

POST /v1/chat/completions

Эта конечная точка обрабатывает запросы на завершение чата.
Полностью совместим с API завершения чата OpenAI

Текст запроса :

свойства OpenAI ---Standard---

- `model` (строка)
  Конкретная модель, используемая для завершения.
- `messages` (массив)
  Журнал бесед в виде списка сообщений.
  - Для каждого сообщения требуется следующее:
    - `role` (строка)
      Роль отправителя сообщения. Должно быть `system`, `user` или `assistant`.

    - `content` (строка)

      Фактический текст сообщения.
- `temperature` (число, необязательно)

  Управляет случайностью, начиная от 0 до 2. Более высокие значения (0,8) создают различные выходные данные, а более низкие значения (0.2) создают ориентированные, согласованные выходные данные.
- `top_p` (число, необязательно)

  Управляет разнообразием выделения маркеров от 0 до 1. Значение 0,1 означает, что считаются только маркеры в верхней% вероятности 10.
- `n` (целое число, необязательно)

  Количество альтернативных вариантов завершения для каждого входного сообщения.
- `stream` (логический, необязательный)

  Если значение истинно, отправляет частичные ответы на сообщения как события, посылаемые сервером, заканчиваясь сообщением `data: [DONE]`.
- `stop` (строка или массив, необязательно)

  До 4 последовательностей, которые приведут к остановке генерации дальнейших токенов моделью.
- `max_tokens` (целое число, необязательно)

  Максимальное количество токенов для генерации. Для более новых моделей используйте `max_completion_tokens` вместо этого.
- `max_completion_token` (целое число, необязательно)

  Максимальное ограничение токенов для генерации, включая видимые выходные данные и токены рассуждения.
- `presence_penalty` (число, необязательно)

  Значение от -2.0 до 2.0. Положительные ценности поощряют модель обсуждать новые темы, наказывая маркеры, которые уже появились.
- `frequency_penalty` (число, необязательно)

  Значение от -2.0 до 2.0. Положительные значения препятствуют повторению путем наказания маркеров на основе их частоты в тексте.
- `logit_bias` (карта, необязательно)

  Корректирует вероятность появления определенных маркеров в завершении.
- `user` (строка, необязательно)

  Уникальный идентификатор для конечного пользователя, который помогает отслеживать и предотвращать злоупотребления.
- `functions` (массив, необязательный)

  Доступные функции, для которых модель может создавать входные данные JSON.
    - Каждая функция должна включать:

    - `name` (строка)

      Имя функции.
    - `description` (строка)

      Описание функции.
    - `parameters` (объект)

      Параметры функции, описанные как объект схемы JSON.
- `function_call` (строка или объект, необязательный)

  Определяет, как модель реагирует на вызовы функций.
  - Если объект, может включать:
    - `name` (строка, необязательно)

      Имя вызываемой функции.
    - `arguments` (объект, необязательный)

      Аргументы, которые следует передать функции.
- `metadata` (объект, необязательный)

  Словарь пар метаданных "ключ-значение".
- `top_k` (число, необязательно)

  Количество токенов словаря с наивысшей вероятностью, которые нужно сохранить для фильтрации top-k.
- `random_seed` (целое число, необязательно)

  Начальное значение (зерно) для воспроизводимого случайного генератора чисел.
- `ep` (строка, необязательно)

  Изменение поставщика для моделей ONNX. Поддерживает: `"dml"`, `"cuda"`, `"qnn"`, `"cpu"`, `"webgpu"`.
- `ttl` (целое число, необязательно)

  Время хранения модели в оперативной памяти в секундах.
- `tools` (объект, необязательный)

  Инструменты, рассчитанные для запроса.

Текст ответа:

- `id` (строка)

  Уникальный идентификатор завершения чата.
- `object` (строка)

  Тип объекта всегда `"chat.completion"`.
- `created` (целое число)

  Метка времени создания в секундах эпохи.
- `model` (строка)

  Модель, используемая для завершения.
- `choices` (массив)

  Список вариантов завершения, каждый из которых содержит:

- `index` (целое число)

  Индекс этого выбора.
- `message` (объект)

  Созданное сообщение с:
  - `role` (строка)

    Всегда `"assistant"` для ответов.
  - `content` (строка)

    Фактически созданный текст.
- `finish_reason` (строка)

  Почему генерация остановлена (например, `"stop"`, `"length"`, `"function_call"`).

- `usage` (объект)

  Статистика использования токенов:
  - `prompt_tokens` (целое число)

    Маркеры в запросе.
  - `completion_tokens` (целое число)

    Маркеры в завершении.
  - `total_tokens` (целое число)

    Всего используемых токенов.

Пример:

- Текст запроса

JSON

```
{
  "model": "Phi-4-mini-instruct-generic-cpu",
  "messages": [
    {
      "role": "user",
      "content": "Hello, how are you?"
    }
  ],
  "temperature": 0.7,
  "top_p": 1,
  "n": 1,
  "stream": false,
  "stop": null,
  "max_tokens": 100,
  "presence_penalty": 0,
  "frequency_penalty": 0,
  "logit_bias": {},
  "user": "user_id_123",
  "functions": [],
  "function_call": null,
```

```
  "metadata": {}
}
```

- Текст ответа

JSON

```
{
  "id": "chatcmpl-1234567890",
  "object": "chat.completion",
  "created": 1677851234,
  "model": "Phi-4-mini-instruct-generic-cpu",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "I'm doing well, thank you! How can I assist you today?"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 20,
    "total_tokens": 30
  }
}
```

POST /v1/embeddings

Обрабатывает запросы на генерацию встраиваний.
Совместимость с API внедрения OpenAI

Текст запроса :

- `model` (строка)
  Модель внедрения, используемая (например, `"text-embedding-ada-002"`).
- `input` (строка или массив)
  Входной текст для встраивания. Может быть одной строкой или массивом строк или токенов.
- `encoding_format` (строка, необязательно)
  Формат кодирования (`"base64"` или `"float"`).

Текст ответа:

- `object` (строка)

  Всегда `"list"`.
- `data` (массив)

  Список внедренных объектов, каждый из которых содержит:
  - `object` (строка)

    Всегда `"embedding"`.
  - `embedding` (массив)

    Векторное представление входного текста.
  - `index` (целое число)

    Позиция этого встраивания в входной массив.
- `model` (строка)

  Модель, используемая для создания встраиваний.
- `usage` (объект)

  Статистика использования токенов:
  - `prompt_tokens` (целое число)

    Количество маркеров в запросе.
  - `total_tokens` (целое число)

    Всего используемых токенов.

Пример:

- Текст запроса

```JSON
{
  "model": "qwen_w_embeddings",
  "input": "Hello, how are you?"
}
```

- Текст ответа

```JSON
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "embedding": [0.1, 0.2, 0.3, ...],
      "index": 0
    }
  ],
  "model": "qwen_w_embeddings",
  "usage": {
```

```
    "prompt_tokens": 10,
    "total_tokens": 10
  }
}
```

Пользовательский API

GET /foundry/list

Извлекает список всех доступных локальных моделей Foundry в каталоге.

Ответ.

- `models` (массив)
  Список объектов модели, каждый из которых содержит:
  - `name`: уникальный идентификатор модели.
  - `displayName`: человекочитаемое имя для модели, часто совпадающее с названием.
  - `providerType`: тип поставщика, на котором размещена модель (например, AzureFoundry).
  - `uri`: URI ресурса, указывающий на расположение модели в реестре.
  - `version`: номер версии модели.
  - `modelType`: формат или тип модели (например, ONNX).
  - `promptTemplate`:
    - `assistant`: шаблон ответа помощника.
    - `prompt`: шаблон взаимодействия с помощником пользователя.
  - `publisher`: сущность или организация, опубликовавшая модель.
  - `task`: основная задача модели предназначена для выполнения (например, завершения чата).
  - `runtime`:
    - `deviceType`: тип оборудования, на котором запускается модель (например, ЦП).
    - `executionProvider`: поставщик выполнения, используемый для выполнения модели.
  - `fileSizeMb`: размер файла модели в мегабайтах.
  - `modelSettings`:
    - `parameters`: список настраиваемых параметров для модели.
  - `alias`: альтернативное имя или сокращенное имя модели
  - `supportsToolCalling`: указывает, поддерживает ли модель функцию вызова инструментов.
  - `license`: тип лицензии, в котором распространяется модель.

- `licenseDescription`: подробное описание или ссылка на условия лицензии.
- `parentModelUri`: универсальный код ресурса (URI) родительской модели, от которой производна эта модель.

POST /openai/register (запрос о регистрации)

Регистрирует внешний поставщик моделей для использования с Foundry Local.

Текст запроса :

- `TypeName` (строка)
  Имя поставщика (например, `"deepseek"`)
- `ModelName` (строка)
  Имя модели для регистрации (например, `"deepseek-chat"`)
- `BaseUri` (строка)
  Базовый URI, совместимый с OpenAI для поставщика

Ответ.

- 200 OK (Запрос выполнен успешно)
  Пустой текст ответа

Пример:

- Текст запроса

  JSON

  ```
  {
    "TypeName": "deepseek",
    "ModelName": "deepseek-chat",
    "BaseUri": "https://api.deepseek.com/v1"
  }
  ```

GET /openai/models

Извлекает все доступные модели, включая локальные и зарегистрированные внешние модели.

Ответ.

- 200 OK (Запрос выполнен успешно)
  Массив имен моделей в виде строк.

Пример:

- Текст ответа

```JSON
["Phi-4-mini-instruct-generic-cpu", "phi-3.5-mini-instruct-generic-cpu"]
```

GET /openai/load/{name}

Загружает модель в память для ускорения вывода.

Параметры URI:

- `name` (строка)
  Имя модели для загрузки.

Параметры запроса:

- `unload` (логический, необязательный)
  Следует ли автоматически выгрузить модель после простоя. По умолчанию — `true`.
- `ttl` (целое число, необязательно)
  Время жизни в секундах. Если больше 0, переопределяет `unload` параметр.
- `ep` (строка, необязательно)
  Поставщик исполнения для запуска этой модели. Поддерживает: `"dml"`, `"cuda"`, `"qnn"`, `"cpu"`, `"webgpu"`.
  Если параметр не указан, использует параметры из `genai_config.json`.

Ответ.

- 200 OK (Запрос выполнен успешно)
  Пустой текст ответа

Пример:

- URI запроса

```
GET /openai/load/Phi-4-mini-instruct-generic-cpu?ttl=3600&ep=dml
```

GET /openai/unload/{name}

Выгружает модель из памяти.

Параметры URI:

- `name` (строка)
  Имя модели для выгрузки.

Параметры запроса:

- `force` (логический, необязательный)
  Если `true`, настройка TTL игнорируется и выгружается немедленно.

Ответ.

- 200 OK (Запрос выполнен успешно)
  Пустой текст ответа

Пример:

- URI запроса

```
GET /openai/unload/Phi-4-mini-instruct-generic-cpu?force=true
```

GET /openai/unloadall

Выгрузка всех моделей из памяти.

Ответ.

- 200 OK (Запрос выполнен успешно)
  Пустой текст ответа

GET /openai/loadedmodels

Извлекает список загруженных в данный момент моделей.

Ответ.

- 200 OK (Запрос выполнен успешно)
  Массив имен моделей в виде строк.

Пример:

- Текст ответа

```JSON
["Phi-4-mini-instruct-generic-cpu", "phi-3.5-mini-instruct-generic-cpu"]
```

GET /openai/getgpudevice

Извлекает выбранный в данный момент идентификатор устройства GPU.

Ответ.

- 200 OK (Запрос выполнен успешно)
  Целое число, представляющее текущий идентификатор устройства GPU.

GET /openai/setgpudevice/{deviceId}

Устанавливает активное графическое устройство GPU.

Параметры URI:

- `deviceId` (целое число)
  Идентификатор устройства GPU для использования.

Ответ.

- 200 OK (Запрос выполнен успешно)
  Пустой текст ответа

Пример:

- URI запроса

```
GET /openai/setgpudevice/1
```

POST /openai/download

Загружает модель в локальное хранилище.

> ⓘ **Примечание**

Скачивание моделей может занять значительное время, особенно для больших моделей. Рекомендуется задать высокий срок ожидания для этого запроса, чтобы избежать преждевременного прекращения работы.

Текст запроса :

- `model` (`WorkspaceInferenceModel` объект)
  - `Uri` (строка)
    URI модели для скачивания.
  - `Name` (строка) Имя модели.
  - `ProviderType` (строка, необязательно)
    Тип поставщика (например, `"AzureFoundryLocal"`, `"HuggingFace"`).
  - `Path` (строка, необязательно)
    Удаленный путь, где сохранена модель. Например, в репозитории Hugging Face это будет путь к файлам модели.
  - `PromptTemplate` (`Dictionary<string, string>` необязательно)
    Содержит:
    - `system` (строка, необязательно)
      Шаблон для системного сообщения.
    - `user` (строка, необязательно) Шаблон сообщения пользователя.
    - `assistant` (строка, необязательно)
      Шаблон ответа помощника.
    - `prompt` (строка, необязательно)
      Шаблон взаимодействия с помощником пользователя.
  - `Publisher` (строка, необязательно)
    Издатель модели.
- `token` (строка, необязательно)
  Маркер проверки подлинности для защищенных моделей (GitHub или Hugging Face).
- `progressToken` (объект, необязательный)
  Только для AITK. Маркер для отслеживания хода скачивания.
- `customDirPath` (строка, необязательно)
  Пользовательский каталог загрузок (используется для CLI, не требуется для AITK).
- `bufferSize` (целое число, необязательно)
  Размер буфера загрузки HTTP в КБ. Не влияет на модели NIM или Azure Foundry.
- `ignorePipeReport` (логический, необязательный)
  Если `true`, принуждает передачу отчётов о ходе выполнения через HTTP поток вместо канала. По умолчанию устанавливается `false` для AITK и `true` для Foundry Local.

Ответ потоковой передачи:

Во время загрузки сервер передаёт обновления о ходе в формате:

```
("file name", percentage_complete)
```

Окончательный текст ответа:

- `Success` (логическое)

  Выполняется ли скачивание успешно.
- `ErrorMessage` (строка, необязательно)

  Сведения об ошибке при сбое скачивания.

Пример:

- Основное содержание запроса

```JSON
{
  "model":{
    "Uri": "azureml://registries/azureml/models/Phi-4-mini-instruct-generic-cpu/versions/4",
    "ProviderType": "AzureFoundryLocal",
    "Name": "Phi-4-mini-instruct-generic-cpu",
    "Publisher": "",
    "promptTemplate" : {
      "system": "<|system|>{Content}<|end|>",
      "user": "<|user|>{Content}<|end|>",
      "assistant": "<|assistant|>{Content}<|end|>",
      "prompt": "<|user|>{Content}<|end|><|assistant|>"
    }
  }
}
```

- Поток ответа

```
("genai_config.json", 0.01)
("genai_config.json", 0.2)
("model.onnx.data", 0.5)
("model.onnx.data", 0.78)
...
("", 1)
```

- Окончательный ответ

```JSON
{
  "Success": true,
  "ErrorMessage": null
}
```

GET /openai/status

Извлекает сведения о состоянии сервера.

Текст ответа:

- `Endpoints` (массив строк)
  Конечные точки привязки HTTP-сервера.
- `ModelDirPath` (строка)
  Каталог, в котором хранятся локальные модели.
- `PipeName` (строка)
  Текущее имя сервера NamedPipe.

Пример:

- Текст ответа

```JSON
{
  "Endpoints": ["http://localhost:5272"],
  "ModelDirPath": "/path/to/models",
  "PipeName": "inference_agent"
}
```

POST /v1/chat/completions/tokenizer/encode/count

Подсчитывает токены для заданного чатового запроса на завершение без выполнения вычислений.

Текст запроса :

- Тип содержимого: application/json
- Объект JSON в `ChatCompletionCreateRequest` формате:
  - `model` (строка)
    Модель, используемая для токенизации.

- `messages` (массив)

  Массив объектов сообщений с `role` и `content`.

Текст ответа:

- Тип содержимого: application/json
- Объект JSON со счетчиком маркеров:
  - `tokenCount` (целое число)

    Количество маркеров в запросе.

Пример:

- Текст запроса

```JSON
{
  "messages": [
    {
      "role": "system",
      "content": "This is a system message"
    },
    {
      "role": "user",
      "content": "Hello, what is Microsoft?"
    }
  ],
  "model": "Phi-4-mini-instruct-cuda-gpu"
}
```

- Текст ответа

```JSON
{
  "tokenCount": 23
}
```

Справочник по API каталога

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

Foundry Local позволяет разрабатывать и интегрировать собственную службу каталогов. В этом документе приведены рекомендации по следующим вопросам:

- Формат модели, необходимый для API каталога.
- Формат запроса и ответа, необходимый для интеграции API каталога с Foundry Local.

Формат модели

Файлы модели, размещенные в каталоге моделей, должны находиться в формате Open Neural Network Exchange (ONNX) для работы с Foundry Local. Дополнительные сведения о компиляции моделей Hugging Face и PyTorch в ONNX см. в статье о компиляции моделей hugging Face, которые будут выполняться в статье Foundry Local .

Формат API

Просьба

Служба каталога должна поддерживать конечную точку POST, принимающую текст запроса JSON. Формат запроса для API каталога выглядит следующим образом:

- **Метод**: `POST`
- **Тип контента**: `application/json`

Текст запроса *должен* быть объектом JSON, *принимающим* следующие поля:

- `resourceIds`: массив идентификаторов ресурсов, указывающих запрашиваемые ресурсы.
  - `resourceId`: идентификатор ресурса.

    - `entityContainerType`: тип контейнера сущностей (например, `Registry`, `Workspace` и т. д.).
- `indexEntitiesRequest`: объект, содержащий параметры поиска.
    - `filters`: массив объектов фильтра, указывающий критерии фильтрации результатов поиска.
        - `field`: поле для фильтрации (например, `type`, `kind` и т. д.).
        - `operator`: оператор, используемый для фильтра. Например, `eq` (равно), `ne` (не равно), `gt` (больше), `lt` (меньше), и т. д.
        - `values`: массив значений для сопоставления с полем.
    - `orderBy`: массив полей для упорядочивания результатов.
    - `searchText`: строка для поиска в результатах.
    - `pageSize`: максимальное количество возвращаемых результатов (для разбиения на страницы).
    - `skip`: количество пропускаемых результатов (для разбиения на страницы).
    - `continuationToken`: маркер для разбиения на страницы для продолжения предыдущего запроса.

Фильтруемые поля (необязательно)

Необходимо реализовать API каталога, который *принимает* формат запроса, но *необязательно*, чтобы реализовать фильтрацию на стороне сервера в службе каталога. Не реализация фильтрации на стороне сервера — это быстрый способ реализации службы каталога, но это может оказаться не эффективным способом поиска моделей.

Если вы решили реализовать фильтрацию на стороне сервера, можно использовать следующие поля для фильтрации результатов:

- `type`: тип модели (например, `models`, `datasets` и т. д.).
- `kind`: тип модели (например, `Versioned`, `Unversioned` и т. д.).
- `properties/variantInfo/variantMetadata/device`: тип устройства (например, `cpu`, `gpu` и т. д.).
- `properties/variantInfo/variantMetadata/executionProvider`: поставщик выполнения (например, `cpuexecutionprovider`, `webgpuexecutionprovider` и т. д.).

Пример запроса

Bash

```
curl POST <your-catalog-api-endpoint> \
-H "Content-Type: application/json" \
-d '{
```

```
  "resourceIds": [
    {
      "resourceId": "azureml",
      "entityContainerType": "Registry"
    }
  ],
  "indexEntitiesRequest": {
    "filters": [
      {
        "field": "type",
        "operator": "eq",
        "values": [
          "models"
        ]
      },
      {
        "field": "kind",
        "operator": "eq",
        "values": [
          "Versioned"
        ]
      },
      {
        "field": "properties/variantInfo/variantMetadata/device",
        "operator": "eq",
        "values": [
          "cpu",
          "gpu"
        ]
      },
      {
        "field": "properties/variantInfo/variantMetadata/executionProvider",
        "operator": "eq",
        "values": [
          "cpuexecutionprovider",
          "webgpuexecutionprovider"
        ]
      }
    ],
    "pageSize": 10,
    "skip": null,
    "continuationToken": null
  }
}'
```

Ответ

Ответ от API каталога — это объект JSON, содержащий результаты поиска. Схема ответа выглядит следующим образом:

```
JSON
```

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "indexEntitiesResponse": {
      "type": "object",
      "properties": {
        "totalCount": {
          "type": "integer",
          "description": "The total count of entities."
        },
        "value": {
          "type": "array",
          "description": "An array of LocalModel objects.",
          "items": {
            "$ref": "#/definitions/LocalModel"
          }
        },
        "nextSkip": {
          "type": "integer",
          "description": "The number of items to skip for the next request."
        },
        "continuationToken": {
          "type": "string",
          "description": "A token to continue fetching results."
        },
      }
    }
  },
  "definitions": {
    "LocalModel": {
      "type": "object",
      "properties": {
        "annotations": {
          "type": "object",
          "description": "Annotations associated with the model.",
          "properties": {
            "tags": {
              "type": "object",
              "description": "Tags associated with the annotation.",
              "properties": {
                "author": { "type": "string" },
                "alias": { "type": "string" },
                "directoryPath": { "type": "string" },
                "license": { "type": "string" },
                "licenseDescription": { "type": "string" },
                "promptTemplate": { "type": "string" },
                "task": { "type": "string" }
              }
            },
            "systemCatalogData": {
              "type": "object",
              "properties": {
                "publisher": { "type": "string" },
```

```
                "displayName": { "type": "string" },
              }
            },
            "name": { "type": "string" }
          }
        },
        "properties": {
          "type": "object",
          "description": "Properties of the model.",
          "properties": {
            "name": { "type": "string" },
            "version": { "type": "integer" },
            "alphanumericVersion": { "type": "string" },
            "variantInfo": {
              "type": "object",
              "properties": {
                "parents": {
                  "type": "array",
                  "items": {
                    "type": "object",
                    "properties": {
                      "assetId": { "type": "string" }
                    }
                  }
                },
                "variantMetadata": {
                  "type": "object",
                  "properties": {
                    "modelType": { "type": "string" },
                    "device": { "type": "string" },
                    "executionProvider": { "type": "string" },
                    "fileSizeBytes": { "type": "integer" }
                  }
                }
              }
            }
          }
        },
        "version": {
          "type": "string",
          "description": "The version of the model."
        },
        "assetId": {
          "type": "string",
          "description": "The asset ID of the model."
        }
      }
    }
  }
}
```

Рекомендации и руководство по устранению неполадок для Foundry Local

03.07.2025

> ⓘ **Важно!**
>
> - Foundry Local доступен в предварительной версии. Общедоступные предварительные выпуски предоставляют ранний доступ к функциям, которые находятся в активном развертывании.
> - Функции, подходы и процессы могут изменяться или иметь ограниченные возможности до общедоступной доступности.

В этом документе приведены рекомендации и советы по устранению неполадок для Foundry Local.

Рекомендации по обеспечению безопасности

- Используйте Foundry Local в средах, которые соответствуют политикам безопасности вашей организации.
- При обработке конфиденциальных данных убедитесь, что устройство соответствует требованиям безопасности вашей организации.
- Используйте шифрование дисков на устройствах, где кэшированные модели могут содержать конфиденциальные данные тонкой настройки.

Соображения, связанные с лицензиями

При использовании Foundry Local помните о последствиях лицензирования для выполняемых моделей. Вы можете просмотреть полные условия лицензии модели для каждой модели в каталоге моделей с помощью:

```bash
foundry model info <model> --license
```

Область развертывания в рабочей среде

Foundry Local предназначен для вывода на устройстве, *а не* для распределенных, контейнерных или рабочих развертываний на нескольких компьютерах.

Устранение неполадок

Распространенные проблемы и решения

Развернуть таблицу

Проблема	Возможные причины	Решение
Медленное вывод	Модель только ЦП с большим числом параметров	Используйте варианты модели, оптимизированные для GPU, если они доступны
Сбои загрузки модели	Проблемы, связанные с подключением к сети	Проверьте подключение к Интернету и выполните проверку `foundry cache list` состояния кэша
Не удается запустить службу	Проблемы с портами или разрешениями	Попробуйте или сообщите `foundry service restart` о проблеме с журналами с помощью `foundry zip-logs`
Ошибка NPU в Qualcomm (`Qnn error code 5005: "Failed to load from EpContext model. qnn_backend_manager."`)	Ошибка NPU Qualcomm	Под расследованием
`winget install Microsoft.FoundryLocal --scope machine` сбой с параметром "Текущая конфигурация системы не поддерживает установку этого пакета".	Winget блокирует установку области компьютера MSIX из-за ошибки ОС при использовании API подготовки из упаковаемого контекста.	Вместо этого используйте `Add-AppxProvisionedPackage`. `.msix` Скачайте и ее зависимость, а затем запустите в PowerShell с **повышенными привилегиями**: `Add-AppxProvisionedPackage -Online -PackagePath .\FoundryLocal.msix -DependencyPackagePath .\VcLibs.appx -SkipLicense` Это устанавливает Foundry Local для всех пользователей.

Повышение производительности

Если вы испытываете медленное вывод, рассмотрите следующие стратегии:

- Одновременное выполнение моделей ONNX, предоставляемых в наборе средств ИИ для VS Code, приводит к возникновению спорных ресурсов. Остановите сеанс вывода набора средств ИИ перед запуском Foundry Local.
- Использование ускорения GPU при наличии
- Определите узкие места путем мониторинга использования памяти во время вывода.
- Попробуйте использовать более квантизованные варианты модели (например, INT8 вместо FP16)
- Настройка размеров пакетов для неинтерактивных рабочих нагрузок